AI-ASSISTED CODING

ASSISGNMENT

NAME:V.SIDDARTHA
HALL:2303A52062
BATCH:45

Task 1: Sorting Student Records for Placement Drive
Scenario
SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.
Tasks
1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
o Quick Sort
o Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.
Expected Outcome
• Correctly sorted student records.
• Performance comparison between Quick Sort and Merge Sort.
• Clear output of top-performing students.

PROMPT
Sorting Student Records for Placement Drive

Suggested Copilot prompt:
"Generate a Python program that stores student records (name, roll number, CGPA), sorts by CGPA in descending order using Quick Sort and Merge Sort, benchmarks both on large datasets, and prints the top 10 students

CODE
```
rom __future__ import annotations

from dataclasses import dataclass
from random import randint, uniform
from time import perf_counter

@dataclass(frozen=True)
class Student:
```

```
    name: str
    roll_number: str
    cgpa: float

def quick_sort_students(students: list[Student]) -> list[Student]:
    """Return students sorted by CGPA in descending order using Quick Sort."""
    if len(students) <= 1:
        return students[:]

    pivot = students[randint(0, len(students) - 1)].cgpa
    higher = [s for s in students if s.cgpa > pivot]
    equal = [s for s in students if s.cgpa == pivot]
    lower = [s for s in students if s.cgpa < pivot]
    return quick_sort_students(higher) + equal + quick_sort_students(lower)

def merge_sort_students(students: list[Student]) -> list[Student]:
    """Return students sorted by CGPA in descending order using Merge Sort."""
    if len(students) <= 1:
        return students[:]

    mid = len(students) // 2
    left = merge_sort_students(students[:mid])
    right = merge_sort_students(students[mid:])
    return _merge_desc(left, right)

def _merge_desc(left: list[Student], right: list[Student]) -> list[Student]:
    merged: list[Student] = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i].cgpa >= right[j].cgpa:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
```

```
def generate_student_records(count: int) -> list[Student]:
    records: list[Student] = []
    for i in range(1, count + 1):
        records.append(
            Student(
                name=f"Student_{i}",
                roll_number=f"SRU{20260000 + i}",
                cgpa=round(uniform(5.0, 10.0), 2),
            )
        )
    return records

def benchmark_sort_algorithms(students: list[Student]) -> tuple[float, float]:
    start = perf_counter()
    quick_sorted = quick_sort_students(students)
    quick_time = perf_counter() - start

    start = perf_counter()
    merge_sorted = merge_sort_students(students)
    merge_time = perf_counter() - start

    if [s.cgpa for s in quick_sorted] != [s.cgpa for s in merge_sorted]:
        raise ValueError("Quick Sort and Merge Sort produced different orderings.")

    return quick_time, merge_time

def display_top_students(students: list[Student], top_n: int = 10) -> None:
    print(f"\nTop {top_n} Students by CGPA:")
    print("-" * 45)
    for idx, student in enumerate(students[:top_n], start=1):
        print(
            f"{idx:2}. {student.name:<15} "
            f"Roll: {student.roll_number:<12} CGPA: {student.cgpa:.2f}"
        )

def main() -> None:
    dataset_size = 20_000
    students = generate_student_records(dataset_size)

    quick_time, merge_time = benchmark_sort_algorithms(students)
```

```
    final_sorted = merge_sort_students(students)

    print(f"Dataset size: {dataset_size} student records")
    print(f"Quick Sort runtime: {quick_time:.6f} seconds")
    print(f"Merge Sort runtime: {merge_time:.6f} seconds")
    faster = "Quick Sort" if quick_time < merge_time else "Merge Sort"
    print(f"Faster algorithm for this run: {faster}")

    display_top_students(final_sorted, top_n=10)

if __name__ == "__main__":
    main()
```

OUTPUT

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.3 assign.py/task1.py"
Dataset size: 20000 student records
Quick Sort runtime: 0.019235 seconds
Merge Sort runtime: 0.032840 seconds
Faster algorithm for this run: Quick Sort

Top 10 Students by CGPA:
--------------------------------------------
 1. Student_2321    Roll: SRU20262321  CGPA: 10.00
 2. Student_2661    Roll: SRU20262661  CGPA: 10.00
 3. Student_3035    Roll: SRU20263035  CGPA: 10.00
 4. Student_4490    Roll: SRU20264490  CGPA: 10.00
 5. Student_5106    Roll: SRU20265106  CGPA: 10.00
 6. Student_5370    Roll: SRU20265370  CGPA: 10.00
 7. Student_6507    Roll: SRU20266507  CGPA: 10.00
 8. Student_7968    Roll: SRU20267968  CGPA: 10.00
 9. Student_8541    Roll: SRU20268541  CGPA: 10.00
10. Student_10276   Roll: SRU20270276  CGPA: 10.00
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task 2: Implementing Bubble Sort with AI Comments

- **Task: Write a Python implementation of Bubble Sort.**
- **Instructions:**
- **Students implement Bubble Sort normally.**
- **Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).**
- **Request AI to provide time complexity analysis.**
- **Expected Output:**

• A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

PROMPT

Implementing Bubble Sort with AI Comments

Suggested Copilot prompt:
"Write a Python Bubble Sort implementation with clear inline comments explaining each pass, swap logic, and early-termination optimization. Also provide time complexity analysis (best/average/worst)

CODE

```
def bubble_sort(values: list[int]) -> list[int]:
    """Sort a list in ascending order using Bubble Sort."""
    arr = values[:]
    n = len(arr)

    # Each outer pass places the next largest element at the end.
    for pass_index in range(n - 1):
        swapped = False

        # Inner loop compares adjacent pairs in the unsorted region.
        for i in range(n - 1 - pass_index):
            # If left item is greater, swap to push the larger value right.
            if arr[i] > arr[i + 1]:
                arr[i], arr[i + 1] = arr[i + 1], arr[i]
                swapped = True

        # If no swaps happened, list is already sorted; stop early.
        if not swapped:
            break

    return arr

def print_complexity_analysis() -> None:
    print("Bubble Sort Time Complexity Analysis")
    print("-" * 40)
    print("Best Case:    O(n)    (already sorted, early termination)")
    print("Average Case: O(n^2)  (random order)")
    print("Worst Case:   O(n^2)  (reverse sorted)")
    print("Space:        O(1)    (in-place, excluding copy in this demo)")
```

```
def main() -> None:
    sample = [64, 34, 25, 12, 22, 11, 90]
    sorted_sample = bubble_sort(sample)

    print("Original list:", sample)
    print("Sorted list:  ", sorted_sample)
    print()
    print_complexity_analysis()

if __name__ == "__main__":
    main()
```

OUTPUT

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.3 assign.py/task2.py"
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list:   [11, 12, 22, 25, 34, 64, 90]

Bubble Sort Time Complexity Analysis
----------------------------------------
Best Case:    O(n)    (already sorted, early termination)
Average Case: O(n^2)  (random order)
Worst Case:   O(n^2)  (reverse sorted)
Space:        O(1)    (in-place, excluding copy in this demo)
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task 3: Quick Sort and Merge Sort Comparison

- **Task: Implement Quick Sort and Merge Sort using recursion.**
- **Instructions:**
- **Provide AI with partially completed functions for recursion.**
- **Ask AI to complete the missing logic and add docstrings.**
- **Compare both algorithms on random, sorted, and reverse-sorted lists.**
- **Expected Output:**
- **Working Quick Sort and Merge Sort implementations.**
- **AI-generated explanation of average, best, and worst-case complexities.**

PROMPT

Quick Sort and Merge Sort Comparison

Suggested Copilot prompt:
"Complete recursive Quick Sort and Merge Sort functions in Python, add proper docstrings with best/average/worst-case complexities, and compare performance on random, sorted, and reverse-sorted lists.

CODE

```
from __future__ import annotations

from random import randint
from time import perf_counter

def quick_sort_recursive(values: list[int]) -> list[int]:
    """
    Sort values using recursive Quick Sort.

    Best/Average: O(n log n)
    Worst: O(n^2) when partitions are highly unbalanced
    Space: O(log n) average recursion stack, O(n) worst
    """
    if len(values) <= 1:
        return values[:]

    pivot = values[randint(0, len(values) - 1)]
    left = [v for v in values if v < pivot]
    equal = [v for v in values if v == pivot]
    right = [v for v in values if v > pivot]
    return quick_sort_recursive(left) + equal + quick_sort_recursive(right)

def merge_sort_recursive(values: list[int]) -> list[int]:
    """
    Sort values using recursive Merge Sort.

    Best/Average/Worst: O(n log n)
    Space: O(n) for merge buffers
    """
    if len(values) <= 1:
        return values[:]

    mid = len(values) // 2
    left = merge_sort_recursive(values[:mid])
```

```
    right = merge_sort_recursive(values[mid:])
    return _merge(left, right)

def _merge(left: list[int], right: list[int]) -> list[int]:
    merged: list[int] = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

def benchmark_case(case_name: str, data: list[int]) -> None:
    start = perf_counter()
    quick_result = quick_sort_recursive(data)
    quick_time = perf_counter() - start

    start = perf_counter()
    merge_result = merge_sort_recursive(data)
    merge_time = perf_counter() - start

    assert quick_result == merge_result == sorted(data)

    print(f"{case_name}")
    print(f"  Quick Sort: {quick_time:.6f} s")
    print(f"  Merge Sort: {merge_time:.6f} s")

def main() -> None:
    n = 20_000
    random_list = [randint(1, 1_000_000) for _ in range(n)]
    sorted_list = sorted(random_list)
    reverse_sorted_list = sorted_list[::-1]
```

```
    print(f"Input size per case: {n}")
    benchmark_case("Random list", random_list)
    benchmark_case("Sorted list", sorted_list)
    benchmark_case("Reverse-sorted list", reverse_sorted_list)

if __name__ == "__main__":
    main()
```

OUTPUT

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.3 assign.py/task3.py"
Input size per case: 20000
Random list
  Quick Sort: 0.032144 s
  Merge Sort: 0.029230 s
Sorted list
  Quick Sort: 0.029814 s
  Merge Sort: 0.018182 s
Reverse-sorted list
  Quick Sort: 0.026737 s
  Merge Sort: 0.021900 s
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>

powershell
Python
```

ask 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

Task:

- **Use AI to suggest the most efficient search and sort algorithms for this use case.**
- **Implement the recommended algorithms in Python.**
- **Justify the choice based on dataset size, update frequency, and performance requirements.**

Expected Output:

- **A table mapping operation → recommended algorithm → justification.**
- **Working Python functions for searching and sorting the inventory.**

PROMPT

Real-Time Application - Inventory Management System

Suggested Copilot prompt:

"Recommend efficient search and sort algorithms for a retail inventory system

with thousands of products. Implement Python functions to search by product ID/name and sort by price/quantity, then justify each algorithm choice.

CODE

```
from __future__ import annotations

from dataclasses import dataclass

@dataclass(frozen=True)
class Product:
    product_id: str
    name: str
    price: float
    quantity: int

def build_inventory_indexes(products: list[Product]) -> tuple[dict[str, Product], dict[str, list[Product]]]:
    """Create hash-map indexes for O(1) average lookup by ID and fast name lookup."""
    id_index: dict[str, Product] = {}
    name_index: dict[str, list[Product]] = {}

    for product in products:
        id_index[product.product_id] = product
        key = product.name.strip().lower()
        name_index.setdefault(key, []).append(product)

    return id_index, name_index

def search_by_id(product_id: str, id_index: dict[str, Product]) -> Product | None:
    return id_index.get(product_id)

def search_by_name(name: str, name_index: dict[str, list[Product]]) -> list[Product]:
    return name_index.get(name.strip().lower(), [])

def sort_by_price(products: list[Product], descending: bool = False) -> list[Product]:
    """Use Timsort (Python built-in sorted): O(n log n), excellent practical performance."""
    return sorted(products, key=lambda p: p.price, reverse=descending)

def sort_by_quantity(products: list[Product], descending: bool = True) -> list[Product]:
    return sorted(products, key=lambda p: p.quantity, reverse=descending)
```

```
def print_recommendation_table() -> None:
    print("Operation -> Recommended Algorithm -> Justification")
    print("-" * 85)
    print(
        "Search by Product ID -> Hash Map (dict) -> "
        "O(1) average lookup, ideal for frequent real-time queries."
    )
    print(
        "Search by Name -> Hash Map index (name->list) -> "
        "Fast exact-name retrieval even with duplicate product names."
    )
    print(
        "Sort by Price/Quantity -> Timsort (sorted) -> "
        "O(n log n), highly optimized in CPython for partially ordered data."
    )

def main() -> None:
    inventory = [
        Product("P1001", "Laptop", 74999.0, 12),
        Product("P1002", "Mouse", 599.0, 140),
        Product("P1003", "Keyboard", 1299.0, 85),
        Product("P1004", "Monitor", 10999.0, 25),
        Product("P1005", "Laptop", 68999.0, 8),
    ]

    id_index, name_index = build_inventory_indexes(inventory)

    print_recommendation_table()
    print()

    print("Search by ID P1003:", search_by_id("P1003", id_index))
    print("Search by name 'Laptop':", search_by_name("Laptop", name_index))

    print("\nSorted by price (low to high):")
    for p in sort_by_price(inventory):
        print(f"{p.product_id} {p.name:<10} Price={p.price:.2f} Qty={p.quantity}")

    print("\nSorted by quantity (high to low):")
    for p in sort_by_quantity(inventory):
        print(f"{p.product_id} {p.name:<10} Price={p.price:.2f} Qty={p.quantity}")
```

```
if __name__ == "__main__":
    main()
```

OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.3 assign.py/task4.py"
Operation -> Recommended Algorithm -> Justification
--------------------------------------------------------------------------------
Search by Product ID -> Hash Map (dict) -> O(1) average lookup, ideal for frequent real-time
queries.
Search by Name -> Hash Map index (name->list) -> Fast exact-name retrieval even with duplicat
e product names.
Sort by Price/Quantity -> Timsort (sorted) -> O(n log n), highly optimized in CPython for par
tially ordered data.

Search by ID P1003: Product(product_id='P1003', name='Keyboard', price=1299.0, quantity=85)
Search by name 'Laptop': [Product(product_id='P1001', name='Laptop', price=74999.0, quantity=
12), Product(product_id='P1005', name='Laptop', price=68999.0, quantity=8)]

Sorted by price (low to high):
P1002 Mouse      Price=599.00 Qty=140
P1003 Keyboard   Price=1299.00 Qty=85
P1004 Monitor    Price=10999.00 Qty=25
P1005 Laptop     Price=68999.00 Qty=8
P1001 Laptop     Price=74999.00 Qty=12

Sorted by quantity (high to low):
P1002 Mouse      Price=599.00 Qty=140
P1003 Keyboard   Price=1299.00 Qty=85
P1004 Monitor    Price=10999.00 Qty=25
P1001 Laptop     Price=74999.00 Qty=12
P1005 Laptop     Price=68999.00 Qty=8
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

- **Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).**
- **Implement sorting algorithms to rank stocks by percentage change.**
- **Implement a search function that retrieves stock data instantly when a stock symbol is entered.**
- **Optimize sorting with Heap Sort and searching with Hash Maps.**
- **Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.**

PROMPT

Real-Time Stock Data Sorting and Searching

Suggested Copilot prompt:

"Simulate stock data (symbol, opening, closing), compute percentage change,

rank stocks using Heap Sort, search instantly by symbol using a hash map, and compare performance with sorted() and dict lookups.

CODE

```
from __future__ import annotations

import heapq
from dataclasses import dataclass
from random import uniform
from time import perf_counter

@dataclass(frozen=True)
class StockRecord:
    symbol: str
    opening_price: float
    closing_price: float

    @property
    def percent_change(self) -> float:
        return ((self.closing_price - self.opening_price) / self.opening_price) * 100

def simulate_stock_data(count: int) -> list[StockRecord]:
    stocks: list[StockRecord] = []
    for i in range(1, count + 1):
        symbol = f"STK{i:04d}"
        opening = round(uniform(50.0, 1500.0), 2)
        change_factor = uniform(0.9, 1.1)
        closing = round(opening * change_factor, 2)
        stocks.append(StockRecord(symbol, opening, closing))
    return stocks

def heap_sort_by_change_desc(stocks: list[StockRecord]) -> list[StockRecord]:
    """Sort stocks by percentage gain/loss descending using heap operations."""
    heap = [(-stock.percent_change, idx, stock) for idx, stock in enumerate(stocks)]
    heapq.heapify(heap)

    result: list[StockRecord] = []
    while heap:
        _, _, stock = heapq.heappop(heap)
        result.append(stock)
    return result
```

```python
def build_symbol_map(stocks: list[StockRecord]) -> dict[str, StockRecord]:
    return {stock.symbol: stock for stock in stocks}

def search_stock(symbol: str, symbol_map: dict[str, StockRecord]) -> StockRecord | None:
    return symbol_map.get(symbol.upper())

def benchmark(stocks: list[StockRecord], symbol_map: dict[str, StockRecord]) -> None:
    start = perf_counter()
    heap_sorted = heap_sort_by_change_desc(stocks)
    heap_time = perf_counter() - start

    start = perf_counter()
    std_sorted = sorted(stocks, key=lambda s: s.percent_change, reverse=True)
    std_time = perf_counter() - start

    target_symbol = stocks[len(stocks) // 2].symbol

    start = perf_counter()
    _ = search_stock(target_symbol, symbol_map)
    map_lookup_time = perf_counter() - start

    start = perf_counter()
    _ = next((s for s in stocks if s.symbol == target_symbol), None)
    linear_lookup_time = perf_counter() - start

    assert [s.symbol for s in heap_sorted] == [s.symbol for s in std_sorted]

    print("Performance Comparison")
    print("-" * 40)
    print(f"Heap Sort runtime:      {heap_time:.6f} s")
    print(f"sorted() runtime:       {std_time:.6f} s")
    print(f"Hash map lookup runtime: {map_lookup_time:.9f} s")
    print(f"Linear lookup runtime:   {linear_lookup_time:.9f} s")

    print("\nTrade-offs")
    print("- Heap Sort is efficient for repeated top-k style extraction.")
    print("- sorted() is simpler and highly optimized for full ordering in Python.")
    print("- Hash map lookup is near O(1), ideal for symbol-based real-time search.")

def print_top_movers(stocks: list[StockRecord], top_n: int = 10) -> None:
```

```
    print(f"\nTop {top_n} Stocks by % Change")
    print("-" * 45)
    for i, stock in enumerate(stocks[:top_n], start=1):
        print(
            f"{i:2}. {stock.symbol:<8} Open={stock.opening_price:8.2f} "
            f"Close={stock.closing_price:8.2f} Change={stock.percent_change:7.2f}%"
        )

def main() -> None:
    stocks = simulate_stock_data(10_000)
    symbol_map = build_symbol_map(stocks)

    ranked_stocks = heap_sort_by_change_desc(stocks)
    print_top_movers(ranked_stocks, top_n=10)

    query = "STK0250"
    result = search_stock(query, symbol_map)
    print(f"\nSearch result for {query}: {result}")

    benchmark(stocks, symbol_map)

if __name__ == "__main__":
    main()
```

OUTPUT

PROBLEMS DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.3 assign.py/task5.py"

Top 10 Stocks by % Change
-------------------------------------------
 1. STK6381  Open=  948.80 Close= 1043.67 Change=  10.00%
 2. STK1447  Open=  153.83 Close=  169.21 Change=  10.00%
 3. STK4585  Open=  476.18 Close=  523.78 Change=  10.00%
 4. STK7915  Open=  437.89 Close=  481.66 Change=  10.00%
 5. STK3989  Open=  682.21 Close=  750.40 Change=  10.00%
 6. STK2497  Open= 1332.00 Close= 1465.07 Change=   9.99%
 7. STK0260  Open= 1323.50 Close= 1455.72 Change=   9.99%
 8. STK4965  Open= 1150.90 Close= 1265.86 Change=   9.99%
 9. STK4845  Open=  246.81 Close=  271.46 Change=   9.99%
10. STK8264  Open=  455.20 Close=  500.66 Change=   9.99%

Search result for STK0250: StockRecord(symbol='STK0250', opening_price=1042.95, closing_price
=1013.94)
Performance Comparison
--------------------------------------
Heap Sort runtime:       0.006113 s
sorted() runtime:        0.001988 s
Hash map lookup runtime: 0.000004700 s
Linear lookup runtime:   0.000114900 s

Trade-offs
- Heap Sort is efficient for repeated top-k style extraction.
- sorted() is simpler and highly optimized for full ordering in Python.
- Hash map lookup is near O(1), ideal for symbol-based real-time search.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

powershell
Python

a reusable Copi

Updated task1.p
files +121 -1

All five task files
plus a Copilot p
each file. I'll run
files next and fi

Run pwsh

```
Set-Locati
"c:\Users\
cuments\De
Assisted c
-m py_comp
3 assign.p
github\12.
py\task2.p
assign.py\
github\12.
py\task4.p
assign.py\
```

Allow Sk

Design p

task5.py:1

Describe what